



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Diseño y Programación de Bases de Datos GT:	Unidad #: 3	Tema: Manejo de seguridades e introducción al PL /SQL
	Fecha:	Tutor:

Introducción:

Bienvenidos al material de lectura 2 de la Unidad #3 de la materia Diseño y Programación de Bases de Datos.

En esta sección se explora la creación de programas en entornos Oracle, centrándose en el uso de SQL*Plus para la implementación de bloques anónimos. Estos bloques permiten ejecutar instrucciones directamente en el servidor sin necesidad de almacenarlas previamente, y ofrecen una forma sencilla y accesible para experimentar con el lenguaje PL/SQL, aprovechando su capacidad para manejar datos, emitir mensajes y controlar el flujo de ejecución mediante estructuras básicas.

Posteriormente, se introduce el concepto de subprogramas almacenados, como procedimientos y funciones, los cuales se compilan y guardan directamente en la base de datos para su reutilización. Esta característica permite una mayor

modularidad y eficiencia, ya que los subprogramas pueden ser invocados repetidamente desde cualquier parte del sistema.

Unidad 3: Manejo de seguridades e introducción al PL /SQL

3.6 Programas.

Para crear nuestros programas podemos utilizar alguna de las herramientas específicas de desarrollo de Oracle, como JDeveloper, que incluyen facilidades gráficas de edición, depuración y compilación. Nosotros hemos optado por utilizar el entorno SQL*Plus, pues está disponible en todas las instalaciones Oracle y permite una plena utilización de todas las características del lenguaje.

3.6.1 Bloques anónimos

Se pueden generar con diversas herramientas, como **SQL*Plus**, y se envían al servidor Oracle, donde serán compilados y ejecutados.

SQL*Plus reconoce el comienzo de un bloque anónimo cuando encuentra la palabra reservada DECLARE o BEGIN. Cuando esto ocurre: limpia el buffer SQL, ignora los «;» y entra en modo INPUT.

```
SQL> BEGIN
2  DBMS_OUTPUT.PUT_LINE('HOLA MUNDO');
3 END;
4 /
```

HOLA MUNDO

Procedimiento PL/SQL terminado con éxito.

El ejemplo anterior muestra un bloque anónimo de código que escribe el texto 'HOLA MUNDO'. Para introducirlo se escribe la palabra reservada BEGIN desde el prompt de SQL*Plus seguida de un retorno de carro. Los números de línea se irán mostrando automáticamente. El símbolo «/» provoca que se guarde el bloque en el buffer y lo envía al servidor para su ejecución.

El bloque anterior carece de las secciones declarativas y de gestión de excepciones, pues la única sección obligatoria es

Actividad Interactiva #1

¿Te ha gustado lo que has leído hasta ahora?

Identifica y resume los puntos clave del texto. Anota ideas principales, términos relevantes y cualquier concepto que consideres esencial:

[illegible]

la ejecutable. Ésta debe contener, al menos, un comando ejecutable, incluso aunque no haga nada tal como aparece en el siguiente ejemplo:

```
SQL> BEGIN
2  NULL;
3 END;
4 /
```

Procedimiento PL/SQL terminado con éxito.

También podemos crear el programa y guardarlo en el buffer SQL, pero sin ejecutarlo automáticamente. Para ello se utiliza un punto después de la última línea de código.

producto_no	Nombre	stock_disponible	unidades_vendidas	precio_actual
12	Articulo 01	19	3	125
24	Articulo 02	23	4	470
43	Articulo 03	65	7	120
70	Articulo 04	46	5	450

Tabla 3.6.1 Tabla Productos.

El siguiente programa muestra el precio de un producto especificado:

```
SQL> DECLARE
2  v_precio NUMBER;
3 BEGIN
4  SELECT precio_actual INTO v_precio
5  FROM productos
6  WHERE PRODUCTO_NO = 70;
7  DBMS_OUTPUT.PUT_LINE(v_precio);
8 END;
9 .
SQL> _
```

Ahora el programa se encuentra cargado en el buffer SQL dispuesto para ser ejecutado utilizando la barra oblicua (/) o el comando RUN.

```
SQL> /
450
```

Procedimiento PL/SQL terminado con éxito.

Podemos guardar el bloque del buffer en un fichero con la orden SAVE según el siguiente formato:

SQL > SAVE nombrefichero [REPLACE]

La opción REPLACE se usará si el fichero ya existía. Para cargar un bloque de un fichero en el buffer SQL se hará mediante el comando GET:

SQL> GET nombrefichero

Una vez cargado se puede ejecutar tal como acabamos de ver con RUN o con «/». También se puede cargar y ejecutar con una sola orden mediante el comando START:

SQL> START nombrefichero

El comando START puede ser sustituido por el símbolo @ con el mismo formato y resultado:

SQL> @nombrefichero

Las variables de sustitución solamente son operativas en los bloques anónimos preferentemente al comienzo y siempre fuera de estructuras condicionales o repetitivas. No podemos usarlas en los procedimientos ya que SQL*Plus realizará la sustitución antes de enviar el bloque al servidor. De esta forma, cuando se compila y almacena el procedimiento se hace con un valor que será siempre el mismo para futuras ejecuciones.

3.7 Subprogramas.

Los procedimientos y funciones se almacenan en la base de datos, donde quedarán disponibles para ser ejecutados, por ello reciben el nombre de **subprogramas almacenados**.

Para su creación nos remitimos a lo expuesto en el apartado anterior para los bloques anónimos teniendo en cuenta que, a diferencia de estos, los procedimientos y funciones, una vez compilados, se almacenan en la base de datos y quedan disponibles para su ejecución mediante los comandos apropiados sin necesidad de volver a cargarlos.

Caso Práctico

Introduciendo estas líneas desde el indicador de SQL*Plus dispondremos de un procedimiento PL/SQL sencillo para consultar los datos de un cliente.

dept_no	nombre	loc
10	CONTABILIDAD	SEVILLA
20	INVESTIGACIÓN	MADRID
30	VENTAS	BARCELONA
40	PRODUCCION	BILBAO

Tabla 3.7.1 Tabla DEPART.

```

SQL> CREATE OR REPLACE
2 PROCEDURE ver_depart (numdepart NUMBER)
3 AS
4   v_dnombre VARCHAR2(14);
5   v_localidad VARCHAR2(14);
6 BEGIN
7   SELECT dnombre, loc INTO v_dnombre, v_localidad
8     FROM depart
9    WHERE dept_no = numdepart;
10    DBMS_OUTPUT.PUT_LINE('Num depart: ' || numdepart || ' * Nombre dep: ' ||
v_dnombre || ' * Localidad: ' || v_localidad);
11 EXCEPTION
12 WHEN NO_DATA_FOUND THEN
13   DBMS_OUTPUT.PUT_LINE('No encontrado departamento ');
14 END ver_depart;
15 /

```

Procedimiento creado.

En el ejemplo anterior podemos observar:

- La **cabecera del procedimiento** (línea 2) indica el nombre del mismo y el parámetro que se utilizará para pasarle valores.
- La palabra **AS** especifica el comienzo del cuerpo del programa y del bloque, comenzando con la zona de declaraciones donde se indicarán los objetos que intervendrán en el programa: variables, constantes, etcétera.
- La palabra **BEGIN** indica el comienzo de la zona de instrucciones.
- La instrucción **SELECT** deposita el resultado de la consulta en las variables v_dnombre, v_localidad que siguen a INTO tal como comentamos al hablar del soporte para consultas del lenguaje. La cláusula WHERE contiene el parámetro con el que se llamó al programa, que será el número de departamento cuyos datos se requieren.
- El procedimiento DBMS_OUTPUT.PUT_LINE visualiza el contenido de las variables y los literales. Para que funcione correctamente la variable SERVEROUTPUT debe estar en ON antes de ejecutar el programa.

- La palabra EXCEPTION indica el comienzo de la zona de tratamiento de excepciones. Esta zona sólo se ejecutará si se produce alguna excepción. Por ejemplo, si al ejecutarse la cláusula SELECT no se encuentra ninguna fila que cumpla la condición, se levantará la excepción NO_DATA_FOUND, se bifurcará el control del programa a esta sección.
- El manejador WHEN NO_DATA_FOUND «caza» la excepción, en el caso de que se produzca, y ejecuta las instrucciones que siguen a la cláusula THEN dando por finalizado el programa.
- El símbolo «/» indica a SQL*Plus al final del procedimiento. Es una abreviatura del comando RUN. Compila y guarda en la base de datos el programa introducido.

Si el compilador detecta errores veremos el siguiente mensaje:

Aviso: Procedimiento creado con errores de compilación.

La orden SHOW ERRORS permite ver los errores detectados. Invocaremos al editor y subsanaremos el error. Hecho esto el programa se compilará y almacenará de nuevo, introduciendo «/» seguido de INTRO.

Ahora, el procedimiento se encuentra almacenado en el servidor de la base de datos y puede ser invocado desde cualquier estación por cualquier usuario autorizado y con cualquier herramienta; por ejemplo, desde SQL*Plus mediante la orden EXECUTE:

```
SQL> SET SERVEROUTPUT ON
SQL> EXECUTE ver_depart(20)
Num depart : 20 * Nombre dep: INVESTIGACIÓN * Localidad: MADRID
```

También podemos invocar al procedimiento desde un bloque PL/SQL, por ejemplo, de esta forma:

```
SQL> BEGIN ver_depart(30); END;
2 /
Num depart: 30 * Nombre dep: VENTAS * Localidad: BARCELONA
```

En realidad, la orden EXECUTE es una utilidad de SQL*Plus que crea un bloque PL/SQL anónimo añadiendo un BEGIN y un END al principio y al final, respectivamente, de la llamada al procedimiento.


```
-- nominal ges_dept(local => localidad, Num_dep => N_departamento);
-- mixta ges_dept(Num_dept, Local => localidad);
...
END;
```

PL/SQL soporta tres tipos de parámetros:

Tipo	Características y utilización
IN	Son parámetros de ENTRADA ; se usan para pasar valores al subprograma. Dentro del subprograma el parámetro actúa como una constante, es decir, no se le puede asignar ningún valor. Por tanto, se sitúa siempre a la derecha del operador de asignación. El parámetro actual puede ser una variable, constante, literal o expresión.
OUT	Son parámetros de SALIDA ; se usan para devolver valores al programa que hizo la llamada. Dentro del subprograma, el parámetro actúa como una variable no inicializada y no puede intervenir en ninguna expresión, salvo para tomar un valor. Se sitúa siempre a la izquierda del operador de asignación. El parámetro actual debe ser una variable.
IN OUT	Son parámetros de ENTRADA/SALIDA ; permiten pasar un valor inicial y devolver un valor actualizado . Dentro del subprograma actúa como una variable inicializada. Puede intervenir en otras expresiones y puede tomar nuevos valores. El parámetro actual debe ser una variable.

Figura 3.7.1 Tipos de parámetros.

El formato genérico de la declaración de cada uno de los parámetros es:

<nombrevariable> [IN | OUT | IN OUT] <tipodedato> [{ := | DEFAULT } <valor>]

Debemos tener en cuenta, además, las siguientes reglas:

- Al indicar los parámetros debemos especificar el tipo, pero no el tamaño.
- En el caso de que el subprograma no tenga parámetros no se pondrán los paréntesis.
- Cuando un subprograma recibe un parámetro en modo OUT y se produce una excepción no tratada, el parámetro actual correspondiente queda sin ningún valor.

Valores por defecto en el paso de parámetros de entrada (modo IN): los parámetros de entrada (todos los que hemos manejado hasta este momento) se pueden inicializar con valores por omisión, es decir, indicando al subprograma que en el caso de que no se pase el parámetro correspondiente, asuma un valor por defecto. Para ello, se utiliza la opción DEFAULT <valor>, o bien := <valor>.

3.7.2 Subprogramas almacenados

Los subprogramas (procedimientos y funciones) que hemos visto hasta ahora se pueden compilar independientemente y almacenar en la base de datos Oracle.

Cuando creamos procedimientos o funciones almacenados desde SQL*Plus utilizando los comandos CREATE PROCEDURE o CREATE FUNCTION, Oracle automáticamente compila el código fuente, genera el código objeto (llamado P-

código) y los guarda en el diccionario de datos. De este modo, quedan disponibles para su utilización.

Los programas almacenados tienen dos estados: disponible (valid) y no disponible (invalid). Si alguno de los objetos referenciados por el programa ha sido borrado o alterado desde la última compilación del programa, quedará en situación de «no disponible» y se compilará de nuevo automáticamente en la próxima llamada. Al compilar de nuevo, Oracle determina si hay que compilar algún otro subprograma referido por el actual. Se puede producir una cascada de compilaciones.

Estos estados se pueden comprobar en la vista USER_OBJECTS:

```
SQL> SELECT OBJECT_NAME, OBJECT_TYPE, STATUS FROM
USER_OBJECTS WHERE OBJECT_NAME='CAMBIAR_OFICIO';
```

OBJECT_NAME	OBJECT_TYPE	STATUS
-----	-----	-----
CAMBIAR_OFICIO	PROCEDURE	VALID

Podemos acceder al código fuente almacenado mediante la vista USER_SOURCE:

```
SQL> SELECT LINE, SUBSTR(TEXT,1,60) FROM USER_SOURCE WHERE
NAME = 'CAMBIAR_OFICIO';
LINE SUBSTR(TEXT,1,60)
```

EMP_NO	APELLIDO	OFICIO	SALARIO	COMISION	DEP_NO
7876	ALONSO	EMPLEADO	1430		20
7902	FERNANDEZ	DIRECTOR	2900		20
7788	GIL	ANALISTA	3900		20
7566	JIMENEZ	DIRECTOR	3867		20
7369	SANCHEZ	EMPLEADO	1040		20
7499	ARROYO	VENDEDOR	2080	390	30
7521	SALA	VENDEDOR	1625	650	30
7654	MARTIN	VENDEDOR	1625	1020	30
7839	REY	PRESIDENTE	6500		10
7698	NEGRO	DIRECTOR	3705		30
7782	CEREZO	DIRECTOR	3185		10

Tabla 3.7.2 Tabla Emple

```

1  PROCEDURE cambiar_oficio (
2      num_empleado NUMBER,      -- En los parámetros ..
3      nuevo_oficio VARCHAR2)    -- ..no se especifica tamaño
4  AS
5      v_anterior_oficio emple.oficio%TYPE;
6  BEGIN
7      SELECT oficio INTO v_anterior_oficio FROM emple
8          WHERE emp_no = num_empleado;
9
10     UPDATE emple SET oficio = nuevo_oficio
11         WHERE emp_no = num_empleado;
12     DBMS_OUTPUT.PUT_LINE(num_empleado||
13         '*Oficio Anterior: '||v_anterior_oficio||
14         '*Oficio Nuevo   : '||nuevo_oficio );
15 END cambiar_oficio;

```

Figura 3.7.2 Procedimiento para cambiar el oficio de un empleado.

Para volver a compilar un subprograma almacenado en la base de datos se emplea la orden ALTER con la opción COMPILE, indicando PROCEDURE o FUNCTION, según el tipo de subprograma:

ALTER { PROCEDURE | FUNCTION } nombresubprograma COMPILE;

Para borrar un subprograma, igual que para eliminar otros objetos, se usa la orden DROP seguida del tipo de subprograma (PROCEDURE o FUNCTION):

DROP { PROCEDURE | FUNCTION } nombresubprograma;

3.7.3 Subprogramas locales

Los subprogramas locales son procedimientos y funciones que se crean dentro de otro subprograma o de un bloque, al final de la sección declarativa:

```

CREATE OR REPLACE pr Ejem1 /* programa ppal. (Contiene el local) */
AS
... /* lista de declaraciones: variables, etc. */
BEGIN
...
sprloc1; /* llamada al subprograma local */
...
END pr Ejem1;

```

Estos subprogramas tienen las siguientes particularidades:

- Se declaran al final de la sección declarativa de otro subprograma o bloque.

- Sólo están disponibles en el bloque en que se crearon y los bloques hijos de éste. Se les aplica las mismas reglas de ámbito y visibilidad que a las variables declaradas en el mismo bloque.
- Se utilizará este tipo de subprogramas cuando no se contemple su reutilización por otros subprogramas (distintos de aquel en el que se declaran).
- En el caso de subprogramas locales con referencias cruzadas o de subprogramas mutuamente recursivos, hay que realizar declaraciones anticipadas, tal como se explica a continuación.

PL/SQL necesita que todos los identificadores, incluidos los subprogramas, estén declarados antes de usarlos. Por eso, la llamada que aparece en el siguiente subprograma es ilegal:

DECLARE

```
...  
PROCEDURE subprograma1 ( ... ) IS  
BEGIN  
...  
Subprograma2( ... ); -- > ERR. identificador no declarado  
...  
END;  
PROCEDURE subprograma2 ( ... ) IS  
BEGIN  
...  
END;  
...
```

Se podía haber invertido el orden de declaración de los procedimientos anteriores, lo cual hubiera resuelto este caso, pero no sirve para procedimientos mutuamente recursivos. El problema se resuelve utilizando declaraciones anticipadas de subprogramas.

```
DECLARE  
PROCEDURE subprograma2 (...); -- declaración anticipada  
PROCEDURE subprograma1 ( ... ) IS  
BEGIN  
Subprograma2( ... ); -- > ahora no dará error.  
...  
END;  
PROCEDURE subprograma2 ( ... ) IS  
BEGIN  
...  
END;  
...
```

3.7.4 Recursividad

PL/SQL implementa, al igual que la mayoría de los lenguajes de programación, la posibilidad de escribir subprogramas recursivos.

```
CREATE OR REPLACE FUNCTION factorial
(n POSITIVE)
RETURN INTEGER -- > devuelve n!
AS
BEGIN
IF n = 1 THEN -- > condición de terminación
RETURN 1;
ELSE
RETURN n * factorial(n - 1); -- > llamada recursiva
END IF;
END factorial;
```

3.8 Funciones

Las funciones tienen una estructura y funcionalidad similar a los procedimientos, pero a diferencia de éstos, las funciones devuelven siempre un valor:

```
FUNCTION <nombredefunción>
[(<lista de parámetros>)]
RETURN <tipo de valor devuelto >
IS
<declaraciones>;
BEGIN
<instrucciones>;
RETURN <expresión>;
...
[EXCEPTION
<excepciones>;]
END [<nombredefunción>;]
```

La lista de parámetros es opcional. Si no hay parámetros no debemos poner paréntesis pues dará error igual que en los procedimientos. Sin embargo, es obligatorio el uso de la cláusula RETURN en la cabecera y el comando RETURN en el cuerpo del programa. No debemos confundirlas:

- La cláusula RETURN de la cabecera especifica el tipo del valor que retorna la función.
- En el cuerpo del programa, el comando RETURN devuelve el control al programa que llamó a la función, asignando el valor de la expresión que sigue al RETURN a la variable que figura en la llamada a la función.

De manera análoga a los procedimientos, para crear o modificar una función utilizaremos el comando CREATE OR REPLACE FUNCTION:

```
CREATE OR REPLACE FUNCTION Encontrar_Num_empleado (  
v_apellido VARCHAR2)  
RETURN REAL  
AS  
N_empleado emple.emp_no%TYPE;  
BEGIN  
SELECT emp_no INTO N_empleado FROM emple  
WHERE apellido = v_apellido;  
RETURN N_empleado;  
END Encontrar_num_empleado;
```

El formato de llamada a una función consiste en utilizarla como parte de una expresión:

<variable> := <nombredefunción> [(listadeparámetros)];

Para invocar a una función desde SQL también tenemos que «hacer algo» con el valor que devuelve, por ejemplo, utilizarlo como parámetro para otra llamada:

```
SQL> BEGIN DBMS_OUTPUT.PUT_LINE(ENCONTRAR_NUM_EMPLEADO  
( 'GIL' ));END;  
2 /  
7788
```

Procedimiento PL/SQL terminado con éxito.

Una función puede tener varios RETURN, pero solo se ejecutará uno de ellos en cada llamada a la función:

```
...  
IF nota < 5 THEN  
RETURN 'SUSPENSO';  
ELSE  
RETURN 'APROBADO';  
END IF;  
...
```

3.9 Procedimientos

Los procedimientos tienen la siguiente estructura general:

```
PROCEDURE <nombreprocedimiento>  
[( <lista de parámetros> )]  
IS
```

Podemos apreciar dos partes:

- La cabecera o especificación del procedimiento. Comienza con la palabra PROCEDURE y termina después de la declaración de parámetros.
- El cuerpo del procedimiento. Corresponde con un bloque PL/SQL. Comienza a continuación de la palabra IS (o AS) y termina con la palabra END, opcionalmente seguida del nombre del procedimiento. En el formato genérico anterior corresponde a la zona sombreada.

Para crear un procedimiento desde SQL*Plus usaremos el siguiente formato:

```
CREATE [OR REPLACE] PROCEDURE <nombreprocedimiento>  
[(listadeparametros)]  
AS ...
```

A continuación, se introducirá el bloque de código PL/SQL sin la palabra DECLARE.

La opción REPLACE actúa en el caso de que hubiese un subprograma almacenado con ese nombre, sustituyéndolo por el nuevo.

En la lista de parámetros se encuentra la declaración de cada uno de los parámetros que se utilizan para pasar valores al programa separados por comas:

(nombreparam1 TIPOP1, nombreparam2 TIPOP2, nombreparam3 TIPOP3, ...)

Podemos invocar al procedimiento desde otro bloque, procedimiento o función. Para ello escribiremos el nombre del procedimiento seguido de la lista de parámetros entre paréntesis, y de un punto y coma:

nombreprocedimientoalquellamamos (listadeparámetros);

La llamada a un procedimiento es una instrucción por sí misma. Cuando se produce la llamada, el control pasa al procedimiento llamado hasta que finaliza su ejecución y el control retorna a la línea siguiente a la llamada.

Por ejemplo, podemos crear un bloque que reciba el apellido y el oficio nuevo. El programa buscará el número de empleado y utilizará una llamada al procedimiento anterior para cambiar oficio:

```
CREATE OR REPLACE PROCEDURE cam_ofi_ (
```

```
v_apellido VARCHAR,  
nue_oficio VARCHAR2)  
IS  
v_n_empleado emple.emp_no%TYPE;  
BEGIN  
SELECT emp_no INTO v_n_empleado FROM emple  
WHERE apellido = v_apellido;  
cambiar_oficio(v_n_empleado, nue_oficio);  
END cam_ofi_;
```

La ejecución del nuevo programa será:

```
SQL> EXECUTE cam_ofi_('FERNANDEZ','ANALISTA');  
7902*Oficio Anterior:DIRECTOR*Oficio Nuevo:ANALISTA  
Procedimiento PL/SQL terminado con éxito.
```

3.10 Cursores

Hasta el momento hemos venido utilizando cursores implícitos. Son muy cómodos y sencillos, pero plantean diversos problemas. Lo más importante es que la subconsulta debe envolver una fila (y sólo una), de lo contrario, se produciría un error. Por ello, dado que normalmente una consulta devolverá varias filas, se suelen manejar cursores explícitos.

3.10.1 Cursores Explícitos

Se utilizan para trabajar con consultas que pueden devolver más de una fila.

Hay cuatro operaciones básicas para trabajar con un cursor explícito:

- 1 **Declaración** del cursor en la zona de declaraciones según el siguiente formato
CURSOR <nombrecursor> IS <sentencia SELECT>;

- 2 **Apertura** del cursor en la zona de instrucciones:

OPEN <nombrecursor>;

La instrucción OPEN ejecuta automáticamente la sentencia SELECT asociada y sus resultados se almacenan en las estructuras internas de memoria manejadas por el cursor.

No obstante, para acceder a la información debemos dar el paso siguiente.

- 3 **Recogida** de información almacenada en el cursor. Se usa el comando FETCH con el siguiente formato:

FETCH <nombrecursor> INTO {<variable>|<listavARIABLES>;}

Después de INTO figurará:

- Una variable que recogerá la información de todas las columnas. Puede declararse de esta forma:

<variable> <nombrecursor>%ROWTYPE;

O una lista de variables. Cada una recogerá la columna correspondiente de la cláusula SELECT; por tanto, serán del mismo tipo que las columnas.

Cada FETCH recupera una fila y el cursor avanza automáticamente a la fila siguiente en cada nueva instrucción FETCH.

- 4 **Cierre del cursor.** Cuando el cursor no se va a utilizar hay que cerrarlo:

CLOSE <nombrecursor>;

El siguiente ejemplo utiliza un cursor explícito para visualizar el nombre y la localidad de todos los departamentos de la tabla DEPART.

```
DECLARE
CURSOR curl IS
SELECT dnombre, loc FROM depart; --1.Declarar
v_nombre VARCHAR2(14);
v_localidad VARCHAR2(14);
BEGIN
OPEN curl; --2.Abrir
LOOP
FETCH curl INTO v_nombre, v_localidad; --3.Recoger
EXIT WHEN curl%NOTFOUND;
DBMS_OUTPUT.PUT_LINE (v_nombre || '*' || v_localidad);
END LOOP;
CLOSE curl; --4.Cerrar
END;
```

El resultado de la ejecución de este bloque será:

```
CONTABILIDAD*SEVILLA
INVESTIGACION*MADRID
VENTAS*BARCELONA
PRODUCCION*BILBAO
```

Observamos que:

- La sentencia SELECT en la declaración del cursor no contiene la cláusula INTO a diferencia de lo que ocurre en los cursores implícitos.

- 3 %ROWCOUNT. Devuelve el número de filas recuperadas hasta el momento por el cursor (número de FETCH realizados satisfactoriamente).
- 4 %ISOPEN. Devuelve verdadero si el cursor está abierto.

La siguiente tabla muestra los valores de retorno de los atributos del cursor en diferentes situaciones:

		%FOUND	%ISOPEN	%NOTFOUND	%ROWCOUNT
OPEN	Antes	Invalid_cursor	F	Invalid_cursor	Invalid_cursor
	Después	NULL	T	NULL	0
PRIMER FETCH	Antes	NULL	T	NULL	0
	Después	T	T	F	1
SIGUIENTES FETCH	Antes	T	T	F	1
	Después	T	T	F	...
ULTIMO FETCH	Antes	T	T	F	N
	Después	F	T	T	N
CLOSE	Antes	F	T	T	N
	Después	Invalid_cursor	F	Invalid_cursor	Invalid_cursor

Tabla 3.10.1 Valores de retorno de los atributos del cursor en diferentes situaciones.

3.10.3 Variables de acoplamiento en el manejo de cursores

La cláusula SELECT del cursor deberá seleccionar frecuentemente las filas de acuerdo con una condición. Cuando se trabaja con SQL interactivo se introducen los términos exactos de la condición. Veamos un ejemplo:

```
SQL> SELECT apellido FROM emple
WHERE dept_no = 20;
```

Pero en un programa PL/SQL los términos exactos de esta condición solamente se conocen en tiempo de ejecución. Esta circunstancia obliga a utilizar un diseño más abierto.

Las variables de acoplamiento cumplen esta función. Su forma de uso suele ser:

- Se declara la variable como cualquier otra.
- Se utiliza la variable en la sentencia SELECT como parte de la expresión.

CURSOR nombrecursor IS clausulaselectconvariableacoplam;

3.10.4 Cursores For ... LOOP

Como ya hemos visto, en muchas ocasiones el trabajo con un cursor consiste en:

- Declarar el cursor.
- Declarar una variable que recogerá los datos del cursor.

- Abrir el cursor.
- Recuperar con FETCH una a una las filas extraídas, introduciendo los datos en la variable, procesándolos y comprobando también si se han recuperado datos o no.
- Cerrar el cursor.

La estructura de cursores FOR...LOOP simplifica estas tareas realizando todas ellas, excepto la declaración del cursor, de manera implícita.

El formato y el uso de esta estructura es:

- Se declara el cursor en la sección declarativa (como cualquier otro cursor).

```
CURSOR <nombrecursor> IS <sentencia SELECT>;
```

- Se procesa el cursor utilizando el siguiente formato:

```
FOR <nombrevareg> IN <nombrecursor> LOOP  
...  
END LOOP;
```

Donde *nombrevareg* es el nombre de la variable de registro que creará el bucle para recoger los datos del cursor.

Al entrar en el bucle:

- Se abre el cursor de manera automática.
- Se declara implícitamente la variable ***nombrevareg*** de tipo ***nombrevareg*** %ROWTYPE y se ejecuta un FETCH implícito, cuyo resultado quedará en ***nombrevareg***.
- A continuación, se realizarán las acciones que correspondan hasta llegar al END LOOP, que sube de nuevo al FOR...LOOP ejecutando el siguiente FETCH implícito, y depositando otra vez el resultado en nombrevareg, y así sucesivamente, hasta procesar la última fila de la consulta. En ese momento, se producirá la salida del bucle y se cerrará automáticamente el cursor.

En el siguiente ejemplo, se visualizan el apellido, el oficio y la comisión de los empleados cuya comisión supera 500 € utilizando CURSOR FOR...LOOP:

```
DECLARE  
CURSOR mi_cursor IS  
SELECT apellido, oficio, comision FROM emple  
WHERE comision > 500;  
BEGIN  
FOR v_reg IN mi_cursor LOOP  
DBMS_OUTPUT.PUT_LINE(v_reg.apellido||'*'||  
v_reg.oficio||'*'||TO_CHAR(v_reg.comision));
```

```
END LOOP;  
END;
```

El resultado será:

```
SALA*VENDEDOR*650  
MARTIN*VENDEDOR*1020
```

Cabe subrayar que la variable ***nombrevareg*** se declara implícitamente y es local al bucle; por tanto, al salir del bucle, la variable de registro no estará disponible.

Dentro del bucle se puede hacer referencia a la variable de registro y a sus campos (cuyo nombre se corresponde con las columnas de la consulta) usando la notación de punto.

